
AI Autoresearch for Chip Design: Autonomous Discovery of a Macro Placement Algorithm

Numaro AI Autoresearch

Abstract

Macro placement determines the floorplan inherited by every later stage of an RTL-to-GDSII flow, yet it remains one of the least automated decisions in physical design. We investigate whether an autonomous research system can discover a placement algorithm under a fixed public benchmark, published baselines, and an evaluator that the system cannot modify. To this end, we present **Relax-and-Gate**, the algorithm produced by a 48-hour autonomous research campaign. Its central idea is a deliberate asymmetry. A smooth, mass-conserving relaxation of the benchmark’s gridded congestion model proposes global moves, while a GPU-batched exact evaluator decides which moves are retained. The relaxation therefore supplies direction without being trusted for the final decision. Relax-and-Gate achieves a mean proxy cost of **0.9366** over 17 circuits, compared with **0.9507** for the published rank-1 result, and produces a legal placement for every circuit. The method was developed on ten circuits, frozen at a named commit, and then evaluated on four held-out circuits. It requires neither a trained placement model nor a labelled placement corpus, and after discovery it runs as ordinary GPU code. We release the placer, placements, verifier, and complete campaign record so that both the result and the process that produced it can be examined.

1 Introduction

Macro placement fixes the locations of SRAMs and hard IP before standard-cell placement and routing. These large blocks shape routing channels, local density, timing, and power delivery. Consequently, a poor floorplan constrains every downstream tool, while a good floorplan leaves those tools room to succeed. Despite this leverage, macro placement is still commonly guided by manual expertise.

Recent learning-based approaches address this problem by producing or refining placements. Examples include learned policies, analytical GPU placers, multimodal placement agents, and systems that steer a base placer [19, 3, 15, 10, 27]. Other systems tune a physical-design flow or evolve a component of an existing optimiser [23, 20, 2, 29, 8]. In these settings, the AI operates inside the design flow and returns a placement, a tuned flow, or a modified component.

We study a complementary setting in which the AI performs the research that creates the placer. The system reads prior work, formulates hypotheses, implements algorithms, runs controlled experiments, and preserves both positive and negative results. This setting raises three challenges.

External validity. An autonomous system can easily overfit an evaluator that it also defines. We therefore use the public Partcl/HRT Macro Placement Challenge [22], with fixed legality rules, published baselines, an external reference evaluator, and a public leaderboard. The system may accelerate this evaluator, but it may not change the quantity being measured.

A discontinuous objective. The benchmark objective is dominated by gridded congestion. Pins are assigned to grid cells using `floor()`, which makes this term piecewise constant with respect to

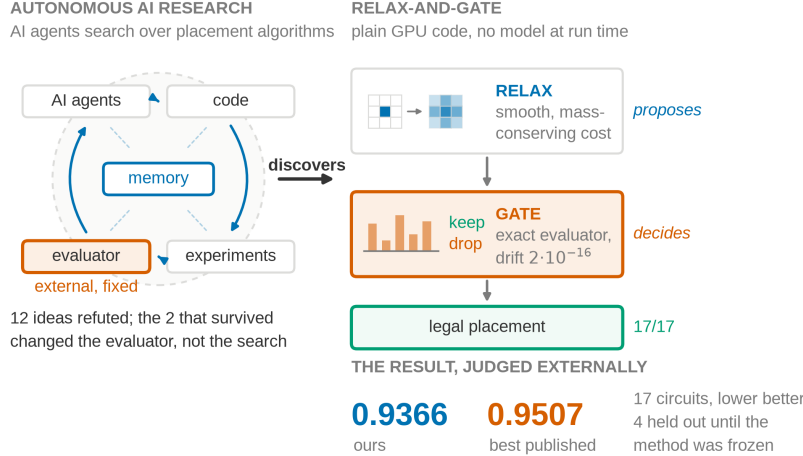


Figure 1: Overview of the research setting and the discovered method. **Left:** the agents form hypotheses, implement and evaluate them, and retain the resulting evidence in a shared memory. The evaluator is external and fixed. **Right:** Relax-and-Gate separates guidance from judgement. A smooth, mass-conserving objective proposes moves, while the exact benchmark objective accepts or rejects them. The resulting placer is ordinary GPU code and contains no language model at run time.

macro coordinates. Direct gradient-based optimisation receives no useful local signal from the term that matters most.

Exact decisions at search scale. A smooth objective can guide global motion, but it is not the benchmark. Conversely, the exact objective is initially too slow to evaluate for a large batch of candidate moves. A successful method must combine the reach of a relaxation with the authority of exact scoring.

To address these challenges, the campaign discovered Relax-and-Gate. The method smooths the grid assignment without changing the total routing demand, then uses the resulting gradients only to propose moves. Every accepted update must independently improve the exact benchmark objective. This separation is the main idea of the paper: *the relaxation provides direction, and the gate provides truth*. A relaxation error can waste computation, but it cannot worsen the retained placement or corrupt a reported score.

Our work makes the following contributions:

- We introduce an autonomous research setting for EDA in which the search unit is a placement algorithm and success is measured by an external evaluator under a frozen protocol.
- We present Relax-and-Gate, which combines a mass-conserving relaxation with drift-free exact GPU acceptance. The construction is simple enough to state precisely and safe even when the relaxation is locally inaccurate.
- We obtain a mean proxy cost of 0.9366 over all 17 circuits, below the published rank-1 mean of 0.9507. The method remains below this result at a $10.6\times$ smaller reported per-circuit budget and shows no observed overfitting on four held-out circuits.
- We release the frozen placer, placements, standalone verifier, and campaign record, including twelve rejected hypotheses, eight detected measurement failures, and one retracted claim.

2 Related work

Algorithm discovery. Learned search has produced improved algorithms for matrix multiplication and sorting [5, 18]. FunSearch and AlphaEvolve pair generative models with programmatic evaluators to discover mathematical constructions and production code [24, 21], while ShinkaEvolve studies the sample efficiency of program evolution [13]. Other systems target the research process itself by generating hypotheses, conducting experiments, or organising specialised research

roles [16, 28, 25, 7]. Our setting differs in that the evaluator, legality rules, baselines, and leaderboard are defined by an independent benchmark, and the output is an executable algorithm for a physical-design problem.

Macro placement. Analytical placers use smooth electrostatic objectives and GPU optimisation [17, 3, 15], with extensions for macros and multi-objective tuning [1]. Hierarchical methods optimise macro groups while accounting for timing and blockages [9]. Learned methods include sequential reinforcement learning [19, 11, 12], black-box optimisation [26], tree search [6], and diffusion [14]. The open benchmark evaluator used in this work was released as part of a reproduction study [4].

Agentic EDA. Existing agentic systems differ mainly in the artifact that the model may change. Placement agents choose coordinates [10, 27]; component-evolution systems rewrite an ordering heuristic or objective [20, 2]; program evolution modifies optimiser source code [29, 8]; and flow agents tune tool parameters [23]. Here, the search unit is a research hypothesis and the delivered artifact is a frozen placer, verifier, and audit trail. Table 1 provides a detailed comparison. AbuPlace is the closest algorithmic comparator because it also uses smooth optimisation, exact acceptance, and incremental scoring. We isolate the differences in Table 2 and do not claim novelty for those shared ingredients.

3 Autonomous research setting

The system operates over a persistent experiment directory rather than a single prompt. It reads the literature and public competitor artifacts, proposes a hypothesis, implements the required code, runs the experiment, and records a verdict. Each hypothesis points to its parent and to the evidence that confirmed or rejected it. Refuted branches therefore remain visible to subsequent iterations instead of being repeatedly rediscovered.

Fixed protocol. Before optimisation, the 17 circuits were divided into ten development, three validation, and four test circuits. A single scoring entry point enforced this split and rejected duplicate experiments. The method and parameters were frozen at a named commit before the four test circuits were evaluated. A standalone verifier imports nothing from the search code and re-derives every final score.

Human and system roles. A human selected the problem and compute budget and reviewed the final result. After launch, the system generated the hypotheses, wrote the method and experiment code, ran the experiments, and assigned the verdicts. The campaign ran for 48 h on one L40S, using approximately 30 h of GPU time and 200 h of CPU time for RTL-to-GDSII validation. No human code edits were made after launch. Language-model token usage was not instrumented, so we do not make a sample-efficiency claim. Tables 4 and 5 provide the full accounting.

4 The discovered method

Relax-and-Gate contains two optimisation stages and one exact measurement instrument. The first stage opens a useful basin with a differentiable relaxation. The second explores that basin using batched discrete moves. Both stages pass through the same exact acceptance gate.

4.1 The exact objective and its GPU implementation

The benchmark assigns each placement the following proxy cost, where lower is better:

$$\text{PROXY} = 1.0 \cdot \text{WL} + 0.5 \cdot \text{Density} + 0.5 \cdot \text{Congestion}, \quad (1)$$

Wirelength is HPWL over continuous pin coordinates and is piecewise linear. Density is computed from continuous macro-to-grid-cell overlap fractions and is also piecewise linear. Congestion is different: the evaluator applies `floor()` to each pin coordinate before routing demand is deposited on the grid. It is therefore piecewise constant. In the initial placements, congestion accounts for 67% of the objective, yet provides no local gradient.

The original Python evaluator processes approximately 0.6 placements per second. The campaign reimplemented it as a GPU-batched tensor program. Routing demand uses constant-time writes into difference arrays followed by cumulative sums, while blockage and density factor into matrix

products along the two axes. Across four structurally different placements for each of the 17 circuits, all 68 comparisons agree with the untouched reference to at most 10^{-14} relative error on every component. Batched throughput reaches approximately 12 000 evaluations per second.

The discrete stage further uses exact incremental scoring. Since a candidate changes one macro, the evaluator retracts only the affected nets at the old location and emits them at the new one. On the largest circuit this updates approximately 104 of 133 186 pins. Throughput increases from 1178 to 61 230 evaluations per second, with a maximum deviation of 2.2×10^{-16} from a full recomputation.

4.2 Relax: preserve mass, remove quantisation

Instead of introducing an unrelated surrogate, the method dequantises the benchmark objective itself. A pin distributes its unit mass bilinearly across neighbouring cells rather than snapping to one cell, and a route contributes according to its continuous overlap with each cell. All other operations, including top- k averaging, capacity normalisation, smoothing, and blockage correction, remain unchanged.

Formally, for a pin at grid-normalised coordinate $u = x/g$ with $k = \lfloor u \rfloor$ and $\alpha = u - k$, the exact model assigns unit demand to cell k ; we assign

$$m_k = 1 - \alpha, \quad m_{k+1} = \alpha, \quad m_k + m_{k+1} = 1. \quad (2)$$

A route occupying the interval $[a, b]$ contributes to cell j its overlap length

$$d_j = |[a, b] \cap [jg, (j+1)g]|, \quad \sum_j d_j = |b - a|. \quad (3)$$

Equations (2) and (3) are what we mean by mass-conserving: demand is redistributed between neighbouring cells but is neither created nor destroyed. The relaxation is not assumed to be unbiased. Its purpose is only to expose a useful direction for global motion, including motion among approximately 2600 macros in one backward pass.

4.3 Gate: trust only the exact objective

For the discrete stage, each round proposes one displacement for each of B macros, screens the candidates for legality, and scores all B candidates exactly in one GPU call. Improving moves are combined after collisions among movers are removed. The combined move is evaluated again and its step is halved until it improves the incumbent.

Writing J for the exact objective and $\tilde{x}_{t+1}$ for a proposal, the acceptance rule is

$$x_{t+1} = \begin{cases} \tilde{x}_{t+1}, & J(\tilde{x}_{t+1}) < J(x_t), \\ x_t, & \text{otherwise,} \end{cases} \quad (4)$$

with J evaluated exactly, never through the relaxation.

This rule creates a one-sided error boundary. If the relaxation is accurate, it proposes useful moves. If it is inaccurate, the exact gate rejects them. Approximation error can therefore cost time, but it cannot increase the retained objective or alter the score reported by the verifier. Figure 2 illustrates why this separation is necessary near convergence.

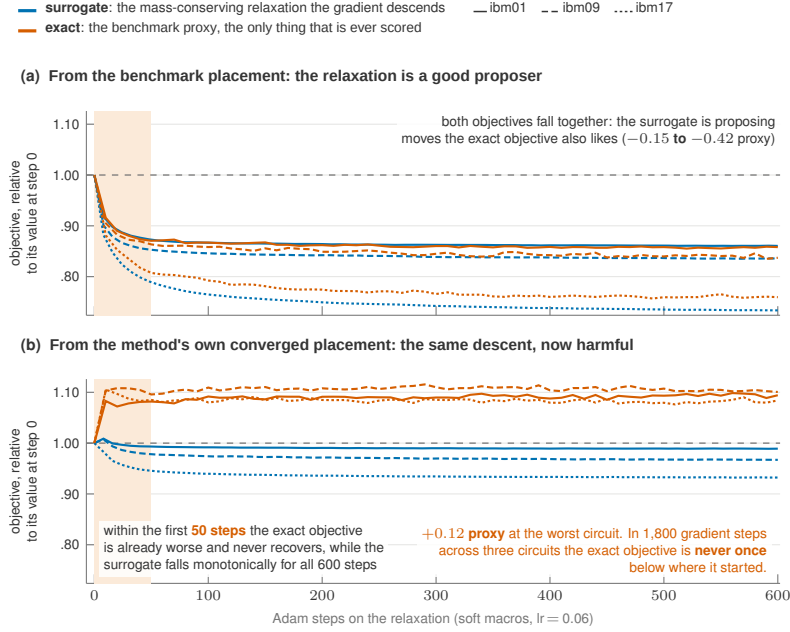


Figure 2: Why the relaxation is used only for proposals. The optimiser, learning rate, and three circuits are fixed while the starting point changes. **(a)** From the benchmark placement, the relaxed and exact objectives initially improve together. **(b)** From a converged Relax-and-Gate placement, the relaxed objective continues to decrease while the exact objective increases within 50 steps. The gate converts this modelling failure into additional run time rather than a worse placement.

5 Results

Unless otherwise stated, we use the frozen method for 900 s per circuit with seed 0 on one L40S. All final numbers are re-derived by the standalone verifier.

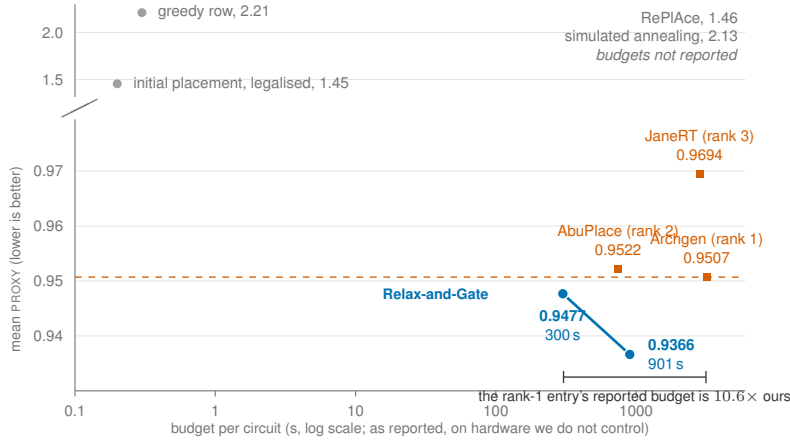
Benchmark quality. Figure 3 presents the main comparison, with exact values in Table 3. At 900 s, Relax-and-Gate achieves a mean proxy cost of 0.936 63, compared with 0.9507 for the published rank-1 entry. At 300 s, its mean is 0.947 67; this remains below the published leader at a $10.6\times$ smaller reported budget. A complete 17-circuit sweep at this budget requires 1.4 h on one GPU.

The leaderboard reports one aggregate number per entry and no dispersion. We therefore report the direct comparison rather than a two-sample significance test. Across three independent 300 s runs, the means are 0.948 08, 0.946 90, and 0.948 03. All three are below 0.9507. Every produced placement passes the benchmark legality checks.

Evaluator agreement. As an end-to-end control, our implementation scores the challenge example placement at 2.2106, compared with the organisers' published value of 2.2109. This is 0.014% relative disagreement. The component-wise tests used for final verification are considerably tighter, as described in Section 4.

Held-out circuits. Relative to a published baseline evaluated on the same circuits, the four test circuits improve by 34.7%, compared with 33.9% on the development circuits. Figure 7 reports every circuit and seed. Since all circuits belong to the same benchmark family, this result supports no observed overfitting within the suite, not generalisation to unrelated industrial designs.

Mean proxy over all 17 circuits against reported budget



Only sweeps covering *all* 17 circuits appear; the bar on the 300 s point is the spread over three independent seeds, every one of which is below the published leader. Budgets are reported figures on hardware we do not control, so this compares *budgets*, not work.

Figure 3: Placement quality against reported per-circuit budget. The vertical axis is broken so that both classical baselines and the competitive region remain visible. Only methods evaluated on all 17 circuits are shown. Relax-and-Gate reaches a mean of 0.947 67 at 300 s and 0.936 63 at 901 s. It is below the published rank-1 mean at both budgets.

6 Analysis of the discovery campaign

The campaign proposed, implemented, and measured twelve modifications to the search strategy. Every modification either lost or finished within ± 0.002 of its paired baseline. By contrast, the two changes that materially improved the final result concerned how the objective was represented and evaluated: the mass-conserving relaxation and exact incremental scoring. At an identical wall-clock budget, the $52\times$ scoring speed-up reduces the mean proxy by 0.025, which is an order of magnitude larger than the effect of any search-side idea.

This result exposes a useful property of the method. Exact gating simplifies the search problem because a proposal mechanism need not be trustworthy; it only needs to produce improving candidates often enough to justify a batch slot. For example, the campaign correctly identified horizontal routing capacity as scarce and derived three direction-aware operators. All three underperformed because their improving-hit rate was too low. A correct structural observation did not imply an efficient proposal distribution.

A second result concerns the unit being placed. Moving only the optimised macros while keeping the cell clusters at their initial positions recovers no gain on the three tested circuits. Moving only the clusters recovers 61 % to 76 %, but the best score still requires joint optimisation. Thus, even on a benchmark named for macro placement, the solution cannot be reduced to macro coordinates alone. Appendix A gives the complete trace.

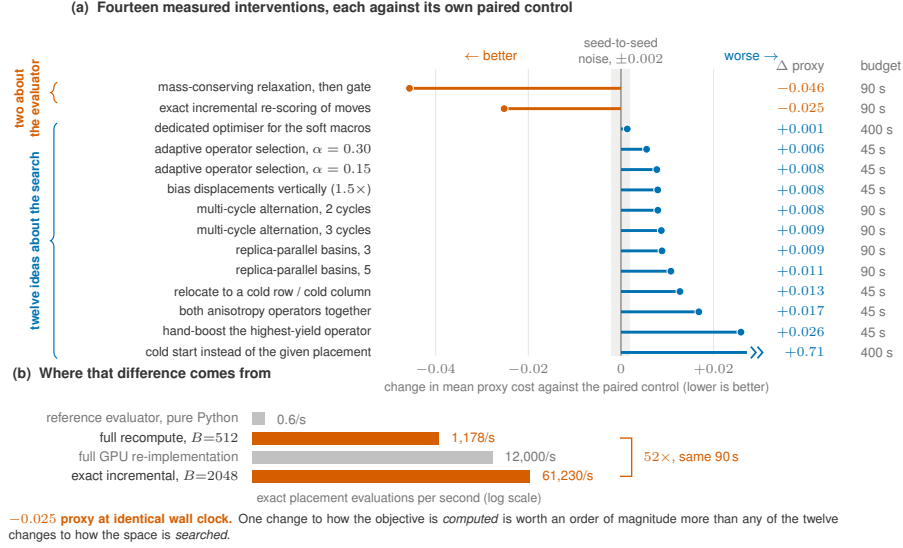


Figure 4: Paired interventions measured during the campaign. Each row changes one component and uses its own control at the same budget on the same three development circuits. Rows are comparable within a group but not across groups. (a) None of the twelve search-side interventions improves its control. (b) Exact incremental evaluation increases scoring throughput and produces the largest quality gain at a fixed wall-clock budget.

7 Audit and reproducibility

The campaign detected eight candidate findings that were caused by measurement failures. Six originated in instrumentation rather than in the underlying hypothesis. The most subtle example was a one-unit-in-the-last-place CUDA division error that moved a grid-aligned pin into an adjacent cell. Random placements did not expose the error; tests constructed at grid boundaries did. Furthermore, both third-party pipelines that failed silently produced results favourable to our method. This pattern motivates a liveness check for every measurement path, namely a value that the pipeline cannot produce when a required stage has been skipped.

The complete list of failures appears in Appendix F. Appendix E also records a timing claim that was retracted after a six-placement analysis showed that its effect was within the method’s own spread. We retain these negative records because they define which claims the final evidence supports.

8 Physical validation

The OpenROAD experiments ask two separate questions: does the proxy locate the same difficult regions as the router, and does a lower proxy predict better design-level outcomes? We lead with *ariane136*, the design whose scored and synthesised netlists match exactly, with 136 of 136 hard macros.

Where does the proxy see the router? Figure 5 compares the cluster-level demand field available to the placer with congestion measured after detailed routing. The spatial correlation is modest, but the hotspot result is more revealing: one quarter of the cells selected by the proxy’s top 5 % also lie in OpenROAD’s top 5 %, a fivefold enrichment over random selection.

The point is not that a cluster-level model reproduces a detailed router. It does not. Rather, a cheap field exposes enough global structure to guide motion. The exact benchmark evaluator then retains authority during search, and OpenROAD assesses the physical consequences afterward.

What improves as the proxy falls? We ran the same open-source physical-design flow from synthesis through detailed routing on six placements of this design. The placements differ only in the amount of search used to produce them. RTL, synthesis, constraints, floorplan, tool build, and machine are fixed, and all six placements route successfully.

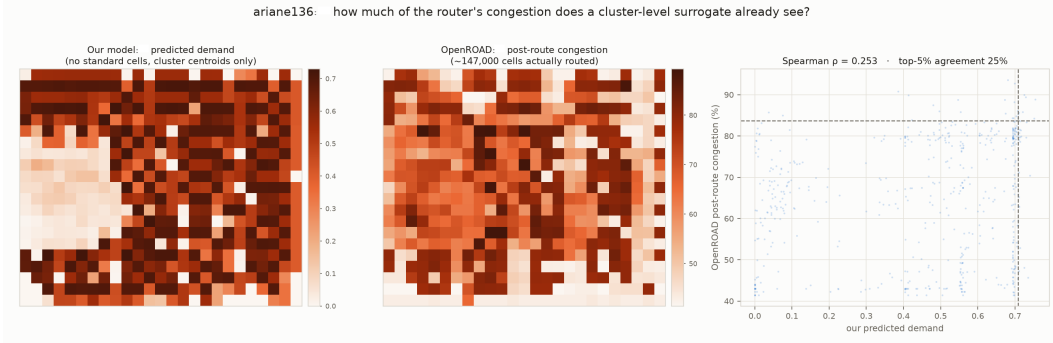


Figure 5: Spatial fidelity of the cheap demand model on ariane136. The left and centre panels compare predicted cluster-level demand with OpenROAD post-route congestion; the right panel compares the corresponding cells. Spearman’s ρ is 0.253, and the two top 5 % sets agree by 25 %, compared with 5 % by chance. The coordinate frames were aligned by an affine map checked against macro locations to within 0.072 μm .

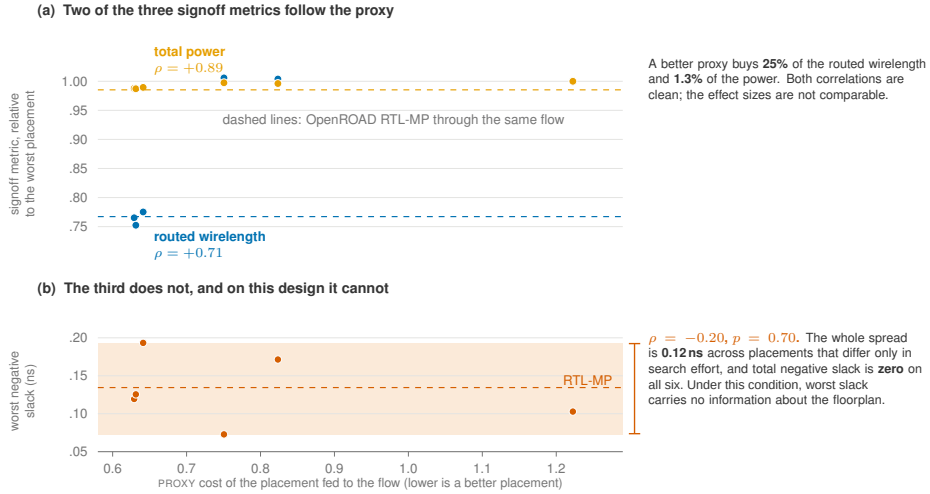


Figure 6: Relationship between benchmark proxy and downstream physical metrics for six placements of one design. All placements use the same flow and differ only in search effort. **(a)** Total power tracks the proxy, while routed wirelength shows a positive but non-significant trend. **(b)** Worst negative slack has no relationship with the proxy; total negative slack is zero for all six placements.

As shown in Figure 6, proxy cost correlates strongly with total power, has a positive but non-significant relationship with routed wirelength, and has no relationship with timing. Only the power correlation is significant at $p < 0.05$. The timing result is expected because total negative slack is zero for all six placements, so this design cannot test a timing hypothesis at the selected clock. The wirelength signal is largely explained by the separation between benchmark-seeded and random-seeded placements; within the better group, the ordering is not monotonic.

These results support a deliberately limited conclusion. The proxy distinguishes poor and good placements and contains useful signal for power, but a lower proxy score does not establish a better chip. The objective omits channel continuity, macro clearance, timing, and power integrity. The full signoff atlas in Figure 8 makes those omitted fields visible, Table 6 quantifies their practical effect, and Figure 9 shows the spatial contrast with OpenROAD’s hierarchical macro placer. Our placement improves some tail metrics but loses on mean congestion and IR drop. The netlist-matching analysis and complete measurements appear in Appendix D.

9 Limitations

The held-out circuits come from the same benchmark family as the development circuits. They test protocol-level overfitting within the suite, not transfer to unrelated industrial designs. Physical

validation is limited to two designs, and the controlled six-placement study covers one design at a clock for which timing does not bind. A stronger evaluation requires several timing-constrained blocks and repeated signoff runs. One published baseline could not be fully reproduced and is quoted as reported. Our final verification is also independent of the search code but remains our own; until the organisers confirm the submitted placements and commit hash, we describe the result as the lowest reported mean to our knowledge rather than as official. Finally, a single campaign cannot establish that the research system transfers to other EDA tasks. It provides one externally scored case study and a reusable experimental record.

10 Conclusion

We investigated whether an autonomous research system can discover an EDA algorithm under an external benchmark and a frozen evaluation protocol. The resulting method, Relax-and-Gate, achieves a mean proxy cost of 0.9366 across 17 circuits while satisfying every legality check. It requires no trained model at run time and was evaluated on held-out circuits only after the method was frozen.

The method is effective because it assigns different responsibilities to approximate and exact computation. A mass-conserving relaxation turns a discontinuous congestion model into a useful global guide, while an exact incremental evaluator retains authority over every accepted update. In short, the relaxation may be wrong, but the placement need not be. This separation makes the algorithm both simple and robust.

The campaign also suggests a broader research lesson. Twelve attempts to improve the search operator failed, while the successful changes improved the representation and evaluation of the objective. Before increasing the complexity of a search strategy, it may be more valuable to ask whether the objective can be made smoother for guidance and cheaper for exact decisions.

References

- [1] Anthony Agnesina, Puranjay Rajvanshi, Tian Yang, Geraldo Pradipta, Austin Jiao, Ben Keller, Brucek Khailany, and Haoxing Ren. AutoDMP: Automated DREAMPlace-based macro placement. In *Proceedings of the 2023 International Symposium on Physical Design (ISPD)*, pages 149–157, 2023. doi: 10.1145/3569052.3578923.
- [2] Ruogu Chen, Weihua Xiao, Ramesh Karri, and Jie Han. CoEvoP&R: Co-evolving placement objectives with routing feedback via large language models. <https://arxiv.org/abs/2607.17398>, 2026.
- [3] Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang, and Lutong Wang. RePlAce: Advancing solution quality and routability validation in global placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 38(9):1717–1730, 2019. doi: 10.1109/TCAD.2018.2859220.
- [4] Chung-Kuan Cheng, Andrew B. Kahng, Sayak Kundu, Yucheng Wang, and Zhiang Wang. Assessment of reinforcement learning for macro placement. In *Proceedings of the 2023 International Symposium on Physical Design (ISPD)*, pages 158–166, 2023. doi: 10.1145/3569052.3578926.
- [5] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. doi: 10.1038/s41586-022-05172-4.
- [6] Zijie Geng, Jie Wang, Ziyang Liu, Siyuan Xu, Zhentao Tang, Mingxuan Yuan, Jianye Hao, Yongdong Zhang, and Feng Wu. Reinforcement learning within tree search for fast macro placement. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 15402–15417, 2024.

- [7] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, Khaled Saab, Dan Popovici, Jacob Blum, Fan Zhang, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amin Vahdat, Pushmeet Kohli, Yossi Matias, Andrew Carroll, Kavita Kulkarni, Nenad Tomasev, Yuan Guan, Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R. D. Costa, José R. Penadés, Yunhan Peng, and Vivek Natarajan. Accelerating scientific discovery with Co-Scientist. *Nature*, 2026. doi: 10.1038/s41586-026-10644-y.
- [8] Taizun Jafri and Vidya A. Chhabria. GR-Evolve: Design-adaptive global routing via LLM-driven algorithm evolution. <https://arxiv.org/abs/2604.22234>, 2026.
- [9] Andrew B. Kahng, Ravi Varadarajan, and Zhiang Wang. Hier-RTLMP: A hierarchical automatic macro placer for large-scale complex IP blocks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(5):1552–1565, 2024. doi: 10.1109/TCAD.2023.3346284.
- [10] Andrew B. Kahng, Sayak Kundu, and Bodhisatta Pramanik. MAGE: Human-like macro placement via agentic multimodal reasoning. <https://arxiv.org/abs/2607.18536>, 2026.
- [11] Yao Lai, Yao Mu, and Ping Luo. MaskPlace: Fast chip placement via reinforced visual representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- [12] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. ChiPFormer: Transferable chip placement via offline decision transformer. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 18346–18364, 2023.
- [13] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. <https://arxiv.org/abs/2509.19349>, 2025.
- [14] Vint Lee, Minh Nguyen, Leena Elzeiny, Chun Deng, Pieter Abbeel, and John Wawrzynek. Chip placement with diffusion models. <https://arxiv.org/abs/2407.12282>, 2025.
- [15] Yibo Lin, Shounak Dhar, Wuxi Li, Haoxing Ren, Brucek Khailany, and David Z. Pan. DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*, 2019. doi: 10.1145/3316781.3317803.
- [16] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. <https://arxiv.org/abs/2408.06292>, 2024.
- [17] Jingwei Lu, Pengwen Chen, Chin-Chih Chang, Lu Sha, Dennis Jen-Hsin Huang, Chin-Chi Teng, and Chung-Kuan Cheng. ePlace: Electrostatics-based placement using fast Fourier transform and Nesterov’s method. *ACM Transactions on Design Automation of Electronic Systems*, 20(2):1–34, 2015. doi: 10.1145/2699873.
- [18] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, Thomas Köppe, Kevin Millikin, Stephen Gaffney, Sophie Elster, Jackson Broshear, Chris Gamble, Kieran Milan, Robert Tung, Minjae Hwang, Taylan Cemgil, Mohammadamin Barekatin, Yujia Li, Amol Mandhane, Thomas Hubert, Julian Schrittwieser, Demis Hassabis, Pushmeet Kohli, Martin Riedmiller, Oriol Vinyals, and David Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964):257–263, 2023. doi: 10.1038/s41586-023-06004-9.
- [19] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. A graph placement methodology for fast chip design. *Nature*, 594(7862):207–212, 2021. doi: 10.1038/s41586-021-03544-w.

- [20] Shibing Mo, Jing Liu, Jianchu Xu, and Ruilin Wu. Order matters: Unveiling the hidden impact of macro placement sequences via proxy-guided LLM evolution. <https://arxiv.org/abs/2606.08904>, 2026. International Conference on Machine Learning (ICML).
- [21] Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. <https://arxiv.org/abs/2506.13131>, 2025.
- [22] Partcl and Hudson River Trading. Partcl/HRT Macro Placement Challenge. <https://github.com/partcl/partcl-macro-placement-challenge-2026>, 2026. Competition repository, benchmark data, scoring rules, and leaderboard.
- [23] Shuo Ren, Zijin Cheng, Yaohui Han, Libo Shen, Leilei Jin, Wanting Tian, Rongliang Fu, Chao Wang, Bei Yu, and Tsung-Yi Ho. AgenticPD: A stage-aware agentic framework for physical design QoR optimization. <https://arxiv.org/abs/2607.04758>, 2026.
- [24] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- [25] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using LLM agents as research assistants. <https://arxiv.org/abs/2501.04227>, 2025.
- [26] Yunqi Shi, Ke Xue, Lei Song, and Chao Qian. Macro placement by wire-mask-guided black-box optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- [27] Ikechukwu Uchendu, Swati Goel, Karly Hou, Ebrahim Songhori, Kuang-Huei Lee, Joe Wenjie Jiang, Vijay Janapa Reddi, and Vincent Zhuang. See it to place it: Evolving macro placements with vision-language models. <https://arxiv.org/abs/2603.28733>, 2026.
- [28] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. <https://arxiv.org/abs/2504.08066>, 2025.
- [29] Xufeng Yao, Jiayi Jiang, Yuxuan Zhao, Peiyu Liao, Yibo Lin, and Bei Yu. EvoPlace: Evolution of optimization algorithms for global placement via large language models. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2026. doi: 10.1109/TCAD.2026.3652945.

A The full campaign trace

This section is the part we would most want a reader to take away, because it is the part that does not depend on our score standing.

Twelve algorithmic ideas, all rejected. The system proposed, implemented and measured adaptive operator selection; congestion-directed and connectivity-directed operators; anisotropic move bias; multi-cycle alternation; basin selection by successive halving; replica-parallel basins; selective padding; reduced-precision screening; symmetry-orbit moves; budget-aware stage splitting; a post-freeze multi-cycle schedule with tail perturbation; and a dedicated optimiser for the unconstrained cluster population. **Every one landed within ± 0.002 of the baseline or lost.**

Two ideas won, and both were about the evaluator. The mass-conserving relaxation and exact incremental scoring. The second is worth quantifying against the twelve: at identical wall clock, the $52\times$ exact speed-up was worth -0.025 , an order of magnitude more than any algorithmic idea produced. **The transferable rule: before buying more search time or a cleverer operator, check whether the evaluator is recomputing what the move did not touch.**

A correct structural fact does not imply a useful operator. The system established that routing capacity is anisotropic and that horizontal is the scarce direction. It then derived three operators directly from that fact (relocate to a cold row, to a cold column, bias displacements vertically) and **all three lost**. In a batched search with an exact accept gate the scarce resource is batch *slots*, and a slot spent on a 3%-hit-rate operator is a slot not spent on a 22%-hit-rate one. The figure of merit is measured improving-hit-rate per slot, not mechanistic plausibility. This is filed under `objective:global-property`, and it is the proposition we would most warn a future campaign about.

Adaptive operator selection lost to a fixed mixture. Probability matching over exactly-measured gain per slot scored *worse* than a fixed near-uniform mixture (0.98582 to 0.99356), and hand-boosting the highest-yield operator was three times worse still. The high-yield operator is easy to improve on early, then self-extinguishes, and it drags the layout toward a wirelength-collapsed basin. The low-yield isotropic operators are buying *exploration*, and greedy credit assignment spends it.

Macro-only coordinate transfer does not preserve the gain. On a benchmark named for macro placement, the natural assumption is that the score comes from where the macros go. On the three circuits we tested it does not. Recombining the halves of an optimised placement with the legalised initial one:

Circuit	initial	optimised	optimised macros only	optimised clusters only
ibm01	1.0381	0.7556	1.0533	0.8676
ibm09	1.1127	0.7523	1.0929	0.9745
ibm17	1.7395	1.1636	1.7456	1.2878

Moving the macros to their optimised coordinates while leaving the cell clusters where they started is worth *nothing*; it is within noise of the initial placement and worse on two of three circuits. Moving only the clusters recovers 61% to 76%. Yet at the joint optimum the macros are not free either: reverting them costs 0.10 to 0.22. Neither population carries the result alone; any method that optimises macro locations against a fixed cell distribution is optimising the wrong object.

The system’s own relaxation is exhausted after its own search. Running the analytical stage from an already-converged placement yields *exactly* zero improvement in 15 of 15 configurations (five learning rates over a $40\times$ range, three circuits, 1400 to 4500 steps). The same optimiser from the initial placement gains 0.15 to 0.44. The mechanism: the surrogate loss keeps falling monotonically while the exact proxy *rises* within fifty steps, because the discrete stage exploits exactly the quantisation the relaxation smooths away in order to have a gradient at all. The two stages are complementary in one direction only. Figure 2 shows the per-step trajectories of both objectives from both starting points; across 1800 gradient steps on three circuits the exact objective is never once below where it started.

B Tables and figures referenced from the main text

Table 1: What the AI is allowed to change. The distinction is not that we modify an algorithm, since program-evolution work does so as well. The distinction is the *scope*: within one campaign the system rewrote the evaluator implementation, the continuous relaxation, the search, the experimental protocol and the validation procedure, and what it delivers is a frozen artifact plus the record needed to check it.

AI role	Representative work	Search unit	Delivered artifact
placement agent	MAGE [10], VeoPlace [27]	coordinates, actions	a placement
component evolution	OrderPlace [20], CoEvoP&R [2]	ordering, objective	an improved component
program evolution	EvoPlace [29], GR-Evolve [8]	source patches	a modified optimiser
flow orchestration	AgenticPD [23]	tool parameters	a tuned flow
research agent	this work	hypotheses, instruments, algorithms, protocols	a frozen placer, its verifier, an audit trail

Table 2: Comparison with the closest algorithmic baseline. We claim novelty for the construction of the relaxation, the drift-free scorer, and the autonomous discovery process. Optimising the exact objective, incremental evaluation, and combining smooth optimisation with exact acceptance are shared with prior work.

	AbuPlace (rank 2)	Relax-and-Gate
Smooth stage	Gaussian and WA-HPWL surrogate	dequantised public objective
Exact acceptance	yes	yes
Incremental scoring	periodic rebuild to control drift	maximum deviation 2.2×10^{-16}
Pipeline	multi-stage cascade	two optimisation stages
Origin	human-designed	autonomous research campaign
Reported mean	0.9522	0.9366, reproduced

Table 3: Mean proxy over the 17 circuits against reported budget. Ours is a hard wall-clock cap on one GPU; leaderboard values and budgets are those published by the competition organisers [22], on hardware we do not control, so this compares *budgets* and not work.

Method	Mean PROXY	Budget/circuit	Provenance
Greedy row (challenge demo)	2.2109	0.3 s	reported
Simulated annealing	2.1251	not reported	reported
RePLAcE	1.4578	not reported	reported
Initial placement, legalised	1.4548	0.2 s	reproduced
JaneRT (rank 3)	0.9694	~ 2843 s	reported
AbuPlace (rank 2)	0.9522	~ 744 s	reported
Archgen (rank 1)	0.9507	~ 3180 s	reported
Relax-and-Gate @ 300 s	0.94767 ± 0.00067	300 s	reproduced
Relax-and-Gate @ 900 s	0.93663	900 s	reproduced

Table 4: Cost of the campaign. GPU- and CPU-hours are approximate; LLM token accounting was not instrumented during the run, which we record as a gap rather than estimate after the fact.

Quantity	Value
Campaign wall-clock	48 h
GPU-hours, discovery + all experiments	≈ 30 h on one L40S
CPU-hours, RTL-to-GDSII flows	≈ 200 h
GPU-hours, final 300 s 17-circuit sweep	1.4 h
LLM calls / tokens / API cost	<i>not instrumented</i>
Human code edits after launch	0

Table 5: The campaign’s 19 verdicts, aggregated by the abstract proposition each tested. The propositions, not the individual experiments, are what transfer to the next problem.

Proposition	What it asserts	W/L
verifier:exact-cheap	an exact objective can be made cheap enough to gate search	2/3
objective:global-property	the objective has exploitable global structure	2/2
objective:deceptive	optimising the stated objective harms an unstated one	2/3
optimum:search-reachable	the optimum is reachable from the given starting point	1/1
relaxation:continuous-exists	a useful continuous relaxation of a discrete objective exists	1/0
record:strong-program-evolver (untagged)	competitors’ public artifacts are worth mining	1/0
	one early verdict predates the proposition scheme	0/1
total		9/10

C Held-out circuit results

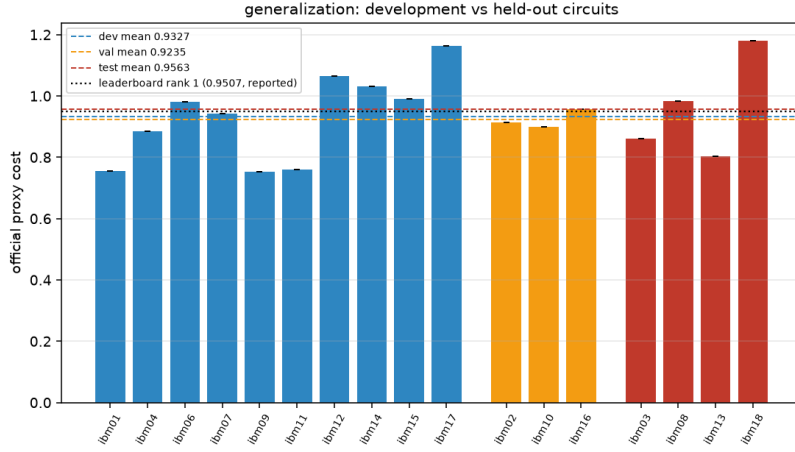


Figure 7: Every circuit, coloured by split. The method was frozen before any held-out circuit (red) was scored. Held-out circuits sit higher in absolute terms only because they are intrinsically harder, since the published baselines also require more on them. The dashed line is the rank-1 entry’s reported mean. Error bars are the spread over three seeds.

D The scored netlist, the built chip, and the export transform

The scored netlist and the built chip are not always the same design. Before reporting any physical number the system checked whether the two describe the same circuit. On one of three designs they do not: its scored netlist declares 133 hard macros while the design the benchmark’s own wrapper synthesises contains 44, a different cache configuration using the same memory macro. All 44 present instances are placed at our coordinates, so that experiment stays internally controlled, but two thirds of the placement is absent from the chip that was built. We therefore lead with the design that matches exactly (136/136).

What the proxy omits, an exact optimiser spends. The proxy has no term for macro clearance, none for channel continuity, and no knowledge that the manufacturable core is larger than its scoring canvas. An optimiser gated on the exact proxy therefore packs tightly, fragments channels, and stays inside a boundary that does not exist downstream; on one design the power-grid inserter could not repair the resulting channels and the flow aborted. Notably the standard open-source hierarchical macro placer *aborts at the same stage with the same error on the same design* and needs a wider halo to proceed, so this failure is not peculiar to proxy-driven optimisation. We therefore define the export transform from published physical constraints only: core area, macro halo, the rules’ minimum channel width, and boundary margin. We freeze this transform rather than fitting it to a reference layout.

Figure 8 follows one placement through the physical fields produced by OpenROAD. The visual continuity between routing demand, routed congestion, and power density explains why the proxy can guide global search. The IR-drop centroid nevertheless lies 318 μm from the macro centroid, in the region occupied by standard cells. The proxy has no power term, so this redistributed power-delivery problem is invisible to its exact gate.

Table 6 summarises the comparison over the routable area. Cells covered by a macro in either placement are excluded, since an unmasked mean would primarily report where the two placers put their macros rather than the physical conditions available to routing.

Relax-and-Gate improves the upper tails of routing demand and cell density, the peaky regions targeted by the proxy’s top- k terms. It also increases mean post-route congestion and worsens IR drop. Figure 9 makes this tension visible: exact optimisation improves the objective it is given, while the downstream maps expose the missing physical terms.

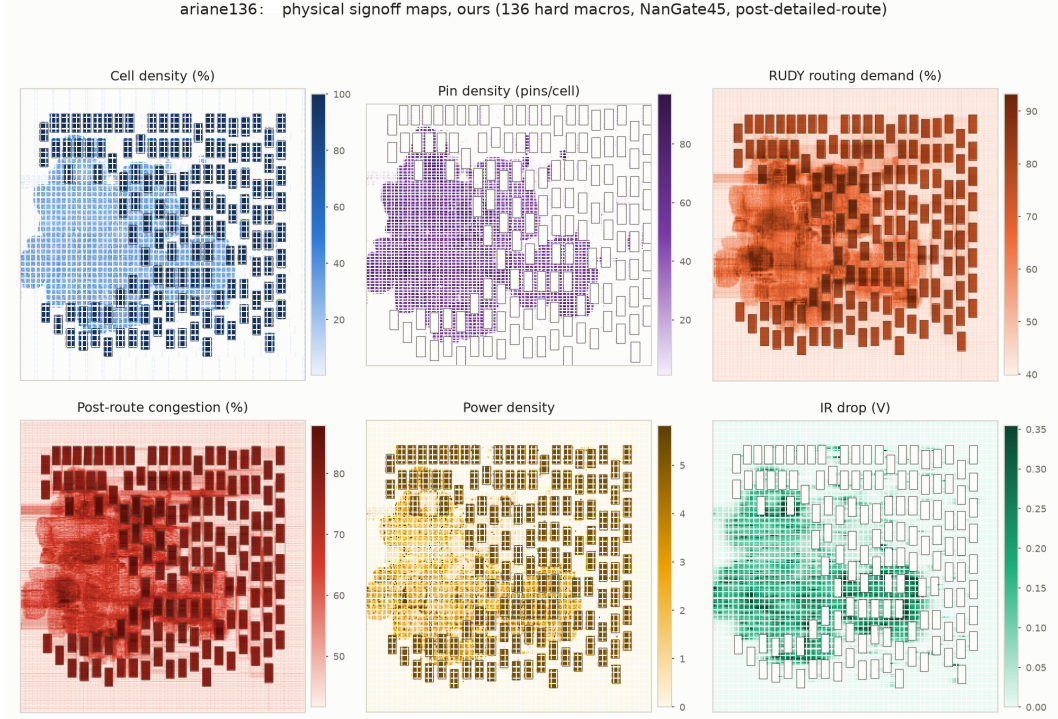


Figure 8: OpenROAD physical fields for our ariane136 placement after detailed routing. The same macro arrangement appears in all six panels, linking cell and pin density to routing demand, routed congestion, power density, and IR drop. Horizontal striping in the routed congestion is already present in the demand model, while the power-integrity field is not.

Table 6: Exploratory signoff comparison over the routable area. RTL-MP required a 10 μm macro halo to pass power-grid insertion, while our placement used the 5 μm flow default. The experiment is therefore a case study rather than a controlled comparison.

Routable-area metric	mean		95th percentile	
	RTL-MP	ours	RTL-MP	ours
Post-route congestion	52.7	54.3	74.5	74.5
Routing demand (RUDY)	52.2	53.5	78.6	77.1
Cell density	24.0	25.0	53.8	48.8
IR drop	51.9	62.4	197	233

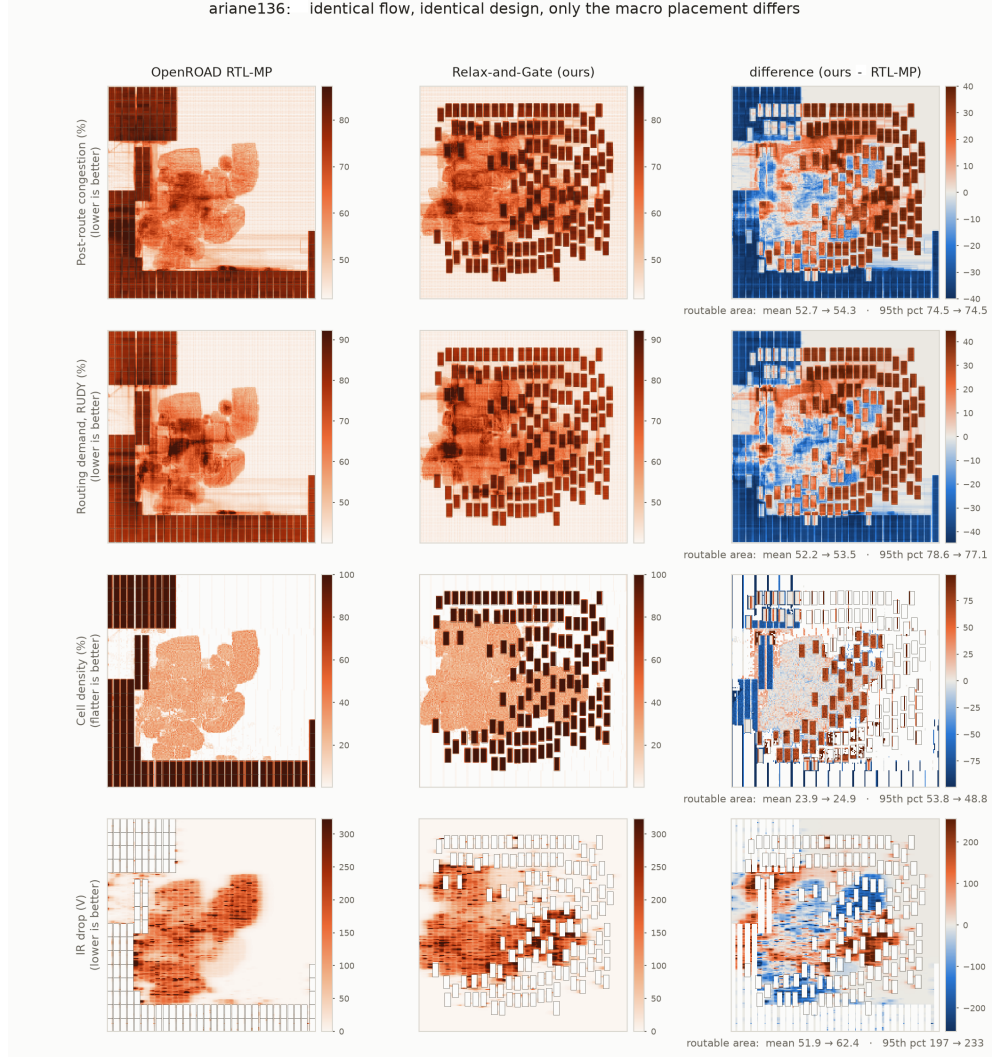


Figure 9: Spatial comparison with OpenROAD RTL-MP on arian136. Columns show RTL-MP, Relax-and-Gate, and their cellwise difference; rows show post-route congestion, routing demand, cell density, and IR drop. RTL-MP banks macros around the periphery and leaves a clear core, whereas Relax-and-Gate distributes macros through it. Because the two placers require different macro halos, this is an exploratory case study rather than a controlled ranking.

E Audit detail

A silent one-ULP error in the evaluator. PyTorch folds `tensor / python_float` into a multiply-by-reciprocal on CUDA, which is not correctly rounded, and that quotient feeds the `floor()` deciding grid-cell membership. One unit in the last place moved a pin into the neighbouring gcell and changed the congestion number, on GPU only, from code that was correct on CPU. Three random test placements matched the reference to 10^{-15} and hid it; it appears only on grid-aligned inputs. Had it survived, every number here would have come from an evaluator that silently disagreed with the official one.

The pattern. Six of the eight errors were failures of *instrumentation* rather than reasoning: a plausible-looking number produced by a broken path. Two involved third-party code that degraded silently instead of crashing, and in both cases the degraded number flattered us. The practical recommendation is that every measurement path needs a *liveness* check, namely a value the pipeline cannot produce if it is broken, in addition to a validity check.

One retraction. An earlier draft of this work reported a 21.3 % worst-slack improvement from the Tier-2 export transform. The ladder of Section 8 shows worst slack varying by 0.12 ns across six of our *own* placements, so that difference sits inside our own spread; the supporting verdict’s own evidence line recorded zero total negative slack, the condition that makes worst slack uninformative. The verdict was retracted in the knowledge base and the claim removed.

F The eight caught measurement errors

1. **A CUDA division that moved a pin across a grid boundary** (Section 7). *Caught by* validating on grid-aligned rather than random inputs.
2. **Benchmark files that disagree with the evaluator.** The shipped preprocessed tensors hardcode unit net weights while the evaluator weights each net by its driver pin’s weight and normalises by a different net count. *Caught by* building the extractor from the evaluator object rather than the convenience file.
3. **An ablation table contaminated by CPU contention.** A background compile inflated early rows including the baseline. *Caught by* measuring the baseline first and last as a drift check.
4. **A baseline whose temperature made it a straw man.** The obvious annealing temperature never improves at all, because typical single-move deltas are $\sim 10^{-4}$. *Caught by* sweeping five decades before publishing the number; the tuned baseline is 0.37 proxy better.
5. **A third-party baseline running with its main stage silently skipped**, because it resolves its netlist by a relative path. Run correctly it scores *worse*, which is itself a finding. *Caught by* an implausibly short runtime.
6. **A comparison that was re-running our own placement** for two and a half hours under another method’s name, because the design config carried both a skip flag and a pointer to our placement file. *Caught by* global-route wirelength identical to the control run to the digit.
7. **A metrics extractor returning zero slack for every design**, because a tool was absent from PATH and the missing key defaulted to zero. *Caught by* a physically impossible constant across designs.
8. **A correlation computed through an unvalidated coordinate transform.** Two frames were paired by sorting each point set independently, which is not a correspondence; it gave a 387 μm “match”. *Caught by* a residual check, which now aborts above 2 μm .